

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«КУБАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «КубГУ»)

Факультет компьютерных технологий и прикладной математики
Кафедра информационных технологий

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 11
по дисциплине
«Операционные системы»

Выполнил студент группы ПМ23 _____ Е.А Шевчук

Направление подготовки 01.03.02 Прикладная математика и информатика
Курс 2

Принял,
канд. техн. наук, доцент каф. ИТ _____ А.А. Полупанов

Краснодар
2026

Задание 1

1. **Перезагрузите тестовую виртуальную машину и в момент её запуска измените команды запуска так, чтобы отключить «тихую» загрузку ОС.** При появлении меню GRUB нажата клавиша `e` для редактирования параметров запуска. В строке, начинающейся с `linux`, удалён параметр `quiet`. На Рисунке 1 показан фрагмент команд запуска после редактирования – параметр `quiet` отсутствует.

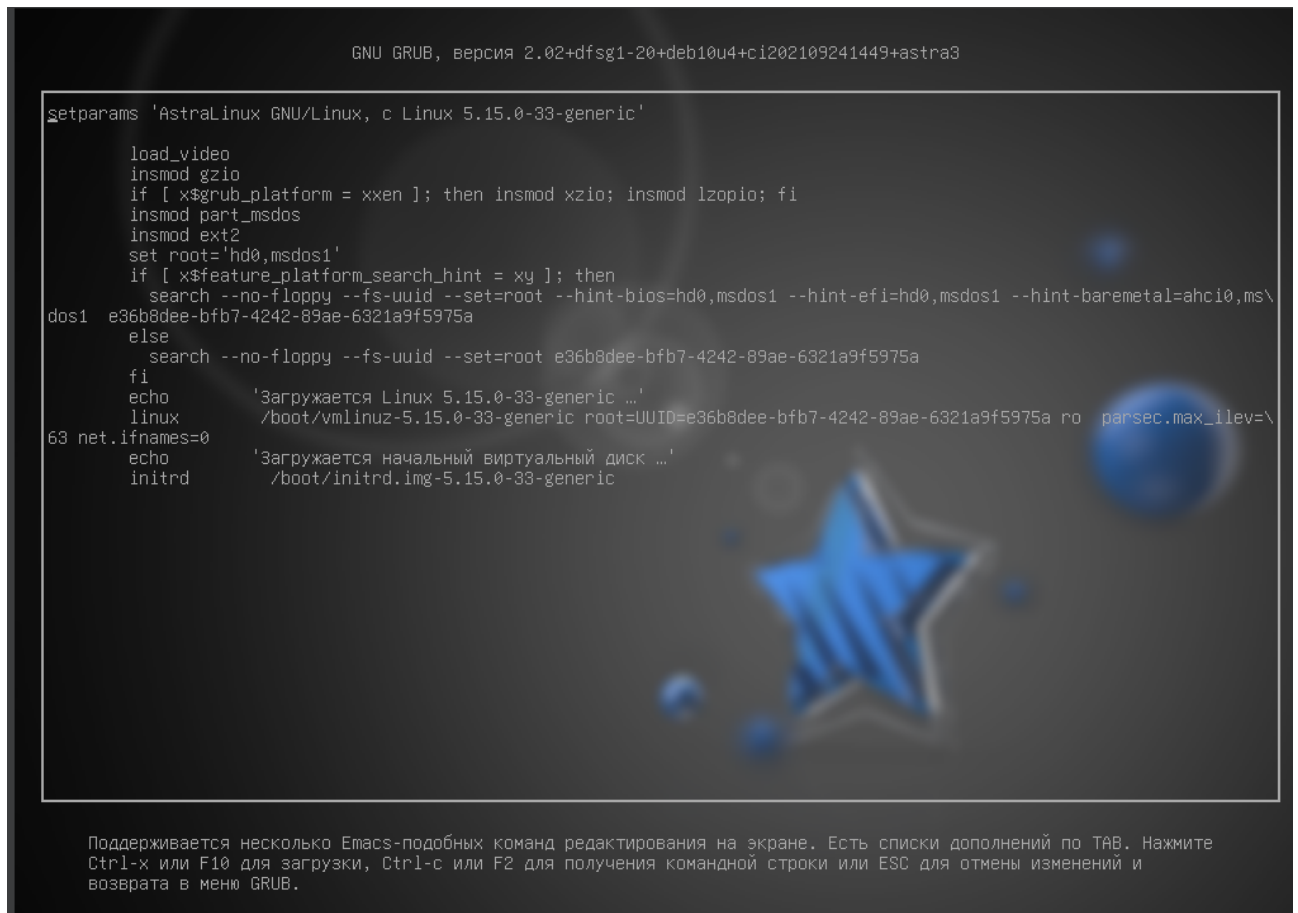


Рисунок 1 – редактирование команд запуска в GRUB для отключения тихой загрузки

2. **Перенастройте загрузчик, чтобы при последующих запусках ОС «тихий» режим загрузки ОС был отключен, а тайм-аут отображения меню загрузки GRUB был 7 секунд. Убедитесь, что настройки применились.**

```
GNU nano 3.2 /etc/default/grub Изменён
# If you change this file, run 'update-grub' afterwards to update
# /boot/grub/grub.cfg.
# For full documentation of the options in this file, see:
#   info -f grub -n 'Simple configuration'

#GRUB_DEFAULT=0
GRUB_TIMEOUT=7
GRUB_DISTRIBUTOR=`lsb_release -i -s 2> /dev/null || echo Debian`
GRUB_CMDLINE_LINUX_DEFAULT="parsec.max_ihev=63 net.ifnames=0"
GRUB_CMDLINE_LINUX=""
GRUB_CMDLINE_LINUX_HARDENED="slub_debug=P page_poison=1 slab_nomerge ption user.ma$
```

Рисунок 2 – содержимое файла /etc/default/grub после изменений

```
sa@astra:~$ sudo nano /etc/default/grub
sa@astra:~$ sudo update-grub
Генерируется файл настройки grub ...
Found background image: /usr/share/images/desktop-base/desktop-grub.png
Найден образ linux: /boot/vmlinuz-5.15.0-33-generic
Найден образ initrd: /boot/initrd.img-5.15.0-33-generic
завершено
sa@astra:~$ █
```

Рисунок 3 – выполнение команды sudo update-grub

3. Выведите на экран целевое состояние системы по умолчанию. Перезагрузите машину, используя целевое состояние systemd.

Перезагрузка выполнена командой sudo systemctl isolate reboot.target.

```
sa@astra:~$ systemctl get-default
graphical.target
sa@astra:~$ sudo systemctl isolate reboot.target █
```

Рисунок 4 – вывод целевого состояния по умолчанию и перезагрузка через systemd

4. Выведите на экран сообщения ядра ОС от начала загрузки и ознакомьтесь с ними.

```
sa@astra:~$ sudo dmesg | less
sa@astra:~$ █
```

Рисунок 5 – запуск просмотра сообщений ядра

```

[ 0.000000] Linux version 5.15.0-33-generic (builder@build5) (gcc (AstraLinuxSE 8.3.0-6) 8.3.0, GNU ld (GNU Binutils for AstraLinux) 2.31.1) #astra2+ci56 SMP Wed Aug 10 15:42:06 UTC 2022 (Astra 5.15.0-33.astra2+ci56-generic 5.15.30)
[ 0.000000] Command line: BOOT_IMAGE=/boot/vmlinuz-5.15.0-33-generic root=UUID=e36b8dee-bfb7-4242-89ae-6321a9f5975a ro parsec.max_ilev=63 net.ifnames=0
[ 0.000000] KERNEL supported cpus:
[ 0.000000]   Intel GenuineIntel
[ 0.000000]   AMD AuthenticAMD
[ 0.000000]   Hygon HygonGenuine
[ 0.000000]   Centaur CentaurHauls
[ 0.000000]   zhaoxin   Shanghai
[ 0.000000] x86/fpu: Supporting XSAVE feature 0x001: 'x87 floating point registers'
[ 0.000000] x86/fpu: Supporting XSAVE feature 0x002: 'SSE registers'
[ 0.000000] x86/fpu: Supporting XSAVE feature 0x004: 'AVX registers'
[ 0.000000] x86/fpu: xstate_offset[2]: 576, xstate_sizes[2]: 256
[ 0.000000] x86/fpu: Enabled xstate features 0x7, context size is 832 bytes, using 'standard' format.
[ 0.000000] signal: max sigframe size: 1776
[ 0.000000] BIOS-provided physical RAM map:
[ 0.000000] BIOS-e820: [mem 0x0000000000000000-0x000000000009fbff] usable
[ 0.000000] BIOS-e820: [mem 0x000000000009fc00-0x000000000009ffff] reserved
[ 0.000000] BIOS-e820: [mem 0x00000000000f0000-0x00000000000fffff] reserved
[ 0.000000] BIOS-e820: [mem 0x0000000000100000-0x00000000001aeffff] usable
[ 0.000000] BIOS-e820: [mem 0x00000000001af000-0x00000000001afffff] ACPI data
[ 0.000000] BIOS-e820: [mem 0x00000000fec00000-0x00000000fec00fff] reserved
[ 0.000000] BIOS-e820: [mem 0x00000000fee00000-0x00000000fee00fff] reserved
[ 0.000000] BIOS-e820: [mem 0x00000000fffc0000-0x00000000ffffffff] reserved
[ 0.000000] NX (Execute Disable) protection: active
[ 0.000000] SMBIOS 2.5 present.
[ 0.000000] DMI: innotek GmbH VirtualBox/VirtualBox, BIOS VirtualBox 12/01/2006
[ 0.000000] Hypervisor detected: KVM
[ 0.000000] kvm-clock: Using msrs 4b564d01 and 4b564d00
[ 0.000000] kvm-clock: cpu 0, msr 1e001001, primary cpu clock
[ 0.000001] kvm-clock: using sched offset of 5976588190345 cycles
[ 0.000004] clocksource: kvm-clock: mask: 0xffffffffffffffff max_cycles: 0x1cd42e4dffb, max_idle_ns: 881590591483 ns
[ 0.000007] tsc: Detected 2496.008 MHz processor
[ 0.001749] e820: update [mem 0x00000000-0x00000fff] usable ==> reserved
[ 0.001752] e820: remove [mem 0x000a0000-0x000fffff] usable
[ 0.001756] last_pfn = 0xa1af0 max_arch_pfn = 0x400000000
[ 0.001814] x86/PAT: Configuration [0-7]: WB WC UC- UC WB WP UC- WT
[ 0.001826] CPU MTRRs all blank - virtualized system.
[ 0.011851] found SMP MP-table at [mem 0x0009fbf0-0x0009fbff]
:

```

Рисунок 6 – фрагмент вывода `dmesg`

Задание 2

1. Если вы не создали модули службы fly-message-monitor.service и resource-demanding1.service (раздел 17.6) в процессе изучения материала модуля, то создайте их, в том числе настройте контрольную группу для службы resource-demanding1.service.

Созданы файлы служб:

```
GNU nano 3.2 /etc/systemd/system/fly-message-monitor.service

[Unit]
Description=Сервис отправки уведомлений на рабочий стол
After=graphical.target
[Service]
ExecStart=/usr/local/bin/fly-message-monitor-sa
Type=simple
Restart=always
RestartSec=0
Environment="DISPLAY=:0" "XAUTHORITY=/home/sa/.Xauthority"
[Install]
WantedBy=graphical.target
```

Рисунок 7 – содержимое файла fly-message-monitor.service

```
GNU nano 3.2 /etc/systemd/system/resource-demanding1.service

[Unit]
Description=Сервис, воспроизводящий высокую нагрузку на CPU
After=graphical.target

[Service]
ExecStart=taskset 0x00000001 /usr/bin/bash /usr/local/bin/resource-demanding1.sh
Type=simple

[Install]
WantedBy=graphical.target
```

Рисунок 8 – содержимое файла resource-demanding1.service

```
GNU nano 3.2 /usr/local/bin/resource-demanding1.sh

#!/bin/bash
/usr/bin/sha256sum /dev/urandom > /dev/null
```

Рисунок 9 – содержимое resource-demanding1.sh

```
sa@astra:~$ sudo mkdir /etc/systemd/system/resource-demanding1.service.d
sa@astra:~$ sudo nano /etc/systemd/system/resource-demanding1.service.d/00-slice.conf
sa@astra:~$ sudo nano /etc/systemd/system/resource-demanding1.service.d/10-CPUSettings.conf
sa@astra:~$ █
```

Рисунок 10 – создание каталога и файлов для контрольной группы

```
GNU nano 3.2 /etc/systemd/system/resource-demanding1.service.d/00-slice.conf

[Service]
MemoryAccounting=yes
CPUsAccounting=yes
```

Рисунок 11 – содержимое 00-slice.conf

```
GNU nano 3.2 /etc/systemd/system/resource-demanding1.service.d/10-CPUSettings.conf

[Service]
CPUQuota=90%
```

Рисунок 12 – содержимое 10-CPUSettings.conf (CPUQuota=90%)

```
sa@astra:~$ sudo chmod +x /usr/local/bin/fly-message-monitor-sa
sa@astra:~$ sudo systemctl start fly-message-monitor.service
sa@astra:~$ sudo systemctl enable fly-message-monitor.service
sa@astra:~$ systemctl status fly-message-monitor.service
• fly-message-monitor.service - Сервис отправки уведомлений на рабочий стол
   Loaded: loaded (/etc/systemd/system/fly-message-monitor.service; enabled; vendor preset:
   Active: active (running) since Thu 2026-04-16 22:15:38 MSK; 13s ago
   Main PID: 5815 (fly-message-mon)
     Tasks: 2 (limit: 2889)
    Memory: 672.0K
    CGroup: /system.slice/fly-message-monitor.service
            └─5815 /bin/bash /usr/local/bin/fly-message-monitor-sa
              └─5822 inotifywait -qq -r -e create /tmp/fly-msg

lines 1-9/9 (END)
```

Рисунок 13 – статус службы fly-message-monitor.service (активна, включена)

2. Создайте службу resource-demanding2.service по аналогии со службой resource-demanding1.service, но так, чтобы скрипт, который она запускает, назывался resource-demanding2.sh и запускался на втором процессоре.

```
GNU nano 3.2 /etc/systemd/system/resource-demanding2.service

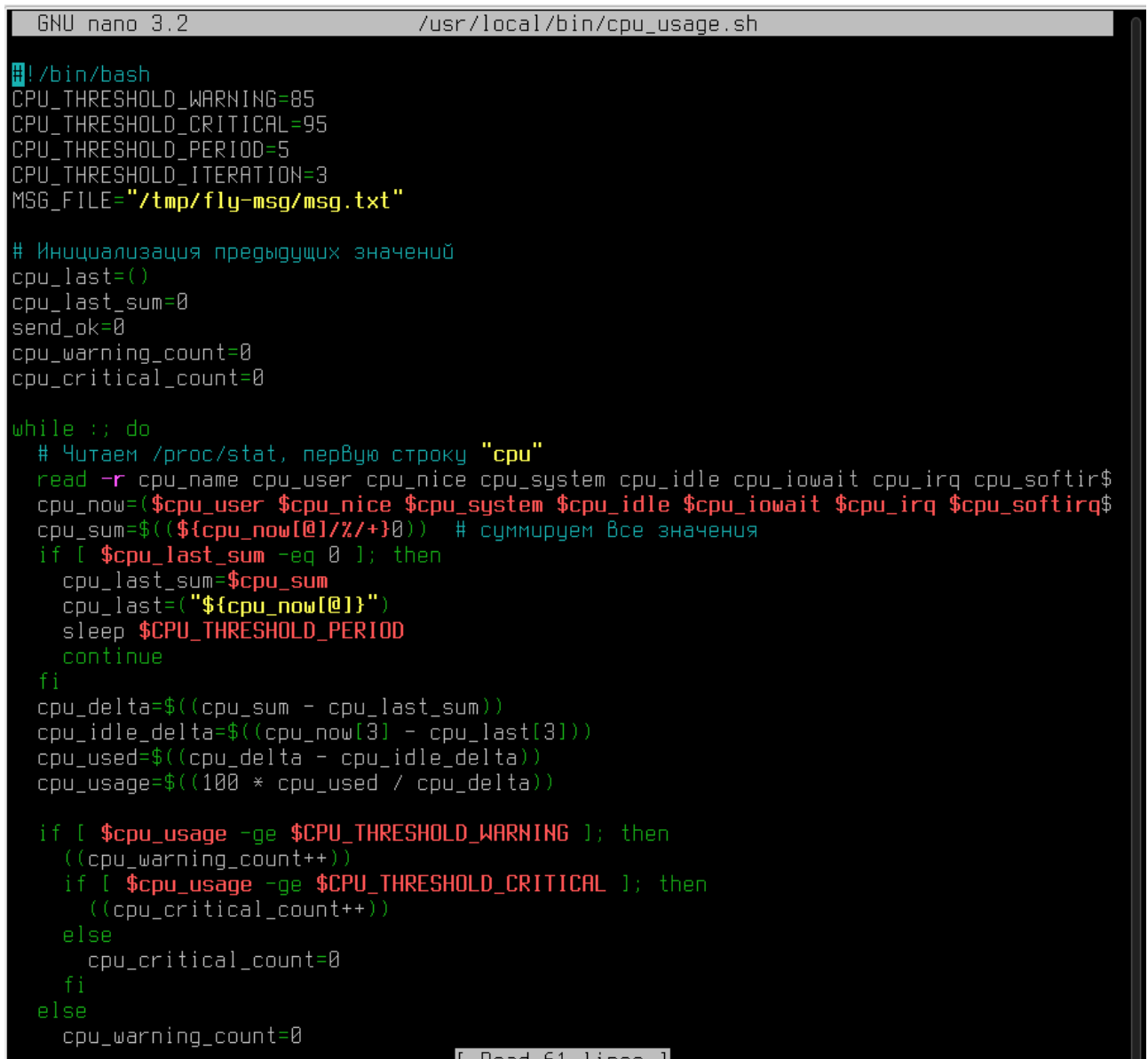
[Unit]
Description=Сервис, Воспроизводящий высокую нагрузку на CPU
After=graphical.target

[Service]
ExecStart=taskset 0x00000002 /usr/bin/bash /usr/local/bin/resource-demanding2.sh
Type=simple

[Install]
WantedBy=graphical.target
```

Рисунок 14 – содержимое файла resource-demanding2.service

3. Создайте скрипт /usr/local/bin/cpu_usage.sh со следующим содержимым (приведено в лабораторной работе). Не забудьте сделать файл исполнимым.



```
GNU nano 3.2 /usr/local/bin/cpu_usage.sh

#!/bin/bash
CPU_THRESHOLD_WARNING=85
CPU_THRESHOLD_CRITICAL=95
CPU_THRESHOLD_PERIOD=5
CPU_THRESHOLD_ITERATION=3
MSG_FILE="/tmp/fly-msg/msg.txt"

# Инициализация предыдущих значений
cpu_last=()
cpu_last_sum=0
send_ok=0
cpu_warning_count=0
cpu_critical_count=0

while ;; do
    # Читаем /proc/stat, первую строку "cpu"
    read -r cpu_name cpu_user cpu_nice cpu_system cpu_idle cpu_iowait cpu_irq cpu_softirq$
    cpu_now=( $cpu_user $cpu_nice $cpu_system $cpu_idle $cpu_iowait $cpu_irq $cpu_softirq$
    cpu_sum=$(( ${cpu_now[@]}%*+)) # суммируем все значения
    if [ $cpu_last_sum -eq 0 ]; then
        cpu_last_sum=$cpu_sum
        cpu_last=( "${cpu_now[@]}" )
        sleep $CPU_THRESHOLD_PERIOD
        continue
    fi
    cpu_delta=$((cpu_sum - cpu_last_sum))
    cpu_idle_delta=$((cpu_now[3] - cpu_last[3]))
    cpu_used=$((cpu_delta - cpu_idle_delta))
    cpu_usage=$((100 * cpu_used / cpu_delta))

    if [ $cpu_usage -ge $CPU_THRESHOLD_WARNING ]; then
        ((cpu_warning_count++))
        if [ $cpu_usage -ge $CPU_THRESHOLD_CRITICAL ]; then
            ((cpu_critical_count++))
        else
            cpu_critical_count=0
        fi
    else
        cpu_warning_count=0
    fi
done
```

Рисунок 15 – содержимое скрипта cpu_usage.sh



```
sa@astra:~$ sudo nano /usr/local/bin/cpu_usage.sh
sa@astra:~$ sudo chmod u+x /usr/local/bin/cpu_usage.sh
sa@astra:~$
```

Рисунок 16 – установка права на выполнение для cpu_usage.sh

4. Создайте модуль службы и назовите его cpu-monitor.service, чтобы он запускал скрипт /usr/local/bin/cpu_usage.sh и стартовал автоматически при загрузке ОС только при цели graphical.target.


```
GNU nano 3.2 /etc/systemd/system/cpu-monitor.service

[Unit]
Description=Мониторинг нагрузки на CPU
After=fly-message-monitor.service

[Service]
ExecStart=/usr/local/bin/cpu_usage.sh
Type=simple

[Install]
WantedBy=graphical.target
```

Рисунок 17 – содержимое файла cpu-monitor.service

5. Выведите на экран содержимое конфигурационного файла модуля cpu-monitor.service, используя systemctl.

```
sa@astra:~$ systemctl cat cpu-monitor.service
# /etc/systemd/system/cpu-monitor.service
[Unit]
Description=Мониторинг нагрузки на CPU
After=fly-message-monitor.service

[Service]
ExecStart=/usr/local/bin/cpu_usage.sh
Type=simple

[Install]
WantedBy=graphical.target
sa@astra:~$
```

Рисунок 18 – вывод содержимого cpu-monitor.service через systemctl cat

6. Отредактируйте файл модуля так, чтобы он запускался только после службы fly-message-monitor.service, используя утилиту systemctl.

```
sa@astra:~$ sudo systemctl edit cpu-monitor.service
sa@astra:~$
```

Рисунок 19 – использование systemctl edit

```
GNU nano 3.2 /etc/systemd/system/cpu-monitor.service.d/.#override.conf18862089079f52ed

[Unit]
After=fly-message-monitor.service
```

Рисунок 20 – в файл `cpu-monitor.service` добавлена директива `After=fly-message-monitor.service`

7. Запустите службу `cpu-monitor.service` и убедитесь, что она работает.

```
sa@astra:~$ sudo systemctl daemon-reload
sa@astra:~$ sudo systemctl enable cpu-monitor.service
Created symlink /etc/systemd/system/graphical.target.wants/cpu-monitor.service → /etc/systemd/system/cpu-monitor.service.
sa@astra:~$ sudo systemctl start cpu-monitor.service
sa@astra:~$ systemctl status cpu-monitor.service
• cpu-monitor.service - Мониторинг нагрузки на CPU
  Loaded: loaded (/etc/systemd/system/cpu-monitor.service; enabled; vendor preset: enabled)
  Drop-In: /etc/systemd/system/cpu-monitor.service.d
           └─override.conf
  Active: active (running) since Thu 2026-04-16 21:28:47 MSK; 9s ago
  Main PID: 2389 (cpu_usage.sh)
    Tasks: 2 (limit: 2889)
  Memory: 612.0K
  CGroup: /system.slice/cpu-monitor.service
          └─2389 /bin/bash /usr/local/bin/cpu_usage.sh
            2395 sleep 5
sa@astra:~$
```

Рисунок 21 – запуск и статус службы `cpu-monitor.service`

8. Переведите систему в целевое состояние `multi-user.target` и убедитесь, что службы `cpu-monitor.service` и `fly-message-monitor.service` не запущены.

```
Astra Linux 1.7.2 astra tty1
astra login: sa
sa
Password:
Secret level [0]: 63
Available secret levels:
  0: for_all
  1: secret
  2: very_secret
  3: very_important
Secret level [0]: 0
Integrity level: 63
Last login: Thu Apr 16 20:07:48 MSK 2026 on :0
sa@astra:~$ systemctl status cpu-monitor.service
• cpu-monitor.service - Мониторинг нагрузки на CPU
  Loaded: loaded (/etc/systemd/system/cpu-monitor.service; enabled; vendor preset: enabled)
  Drop-In: /etc/systemd/system/cpu-monitor.service.d
           └─override.conf
  Active: inactive (dead) since Thu 2026-04-16 21:29:46 MSK; 1min 28s ago
  Process: 2389 ExecStart=/usr/local/bin/cpu_usage.sh (code=killed, signal=TERM)
  Main PID: 2389 (code=killed, signal=TERM)
sa@astra:~$ systemctl status fly-message-monitor.service
• fly-message-monitor.service - Сервис отправки уведомлений на рабочий стол
  Loaded: loaded (/etc/systemd/system/fly-message-monitor.service; disabled; vendor preset: enabled)
  Active: inactive (dead)
lines 1-3/3 (END)
```

Рисунок 22 – статусы служб после перехода в `multi-user.target`

9. Переведите систему в целевое состояние **graphical.target** и убедитесь, что службы **cpu-monitor.service** и **fly-message-monitor.service** запустились. Выполнен возврат в графический режим командой `sudo systemctl isolate graphical.target`. После загрузки графики службы активны

```
sa@astra:~$ sudo systemctl isolate graphical.target
```

Рисунок 23 – переключение в **graphical.target**

10. Запустите службу **resource-demanding1.service**.

```
sa@astra:~$ sudo systemctl start resource-demanding1.service
sa@astra:~$ sudo systemctl start resource-demanding2.service
sa@astra:~$
```

Рисунок 24 – запуск служб **resource-demanding1** и **resource-demanding2**

11. Запустите службу **resource-demanding2.service** и проверьте, что при высокой (более 85%) нагрузке на CPU вы получаете сообщения от скрипта мониторинга.

После запуска обеих ресурсоёмких служб нагрузка на CPU возросла. На Рисунке 25 показано всплывающее уведомление. Сообщение получено от скрипта `cpu_usage.sh` через службу **fly-message-monitor.service**.

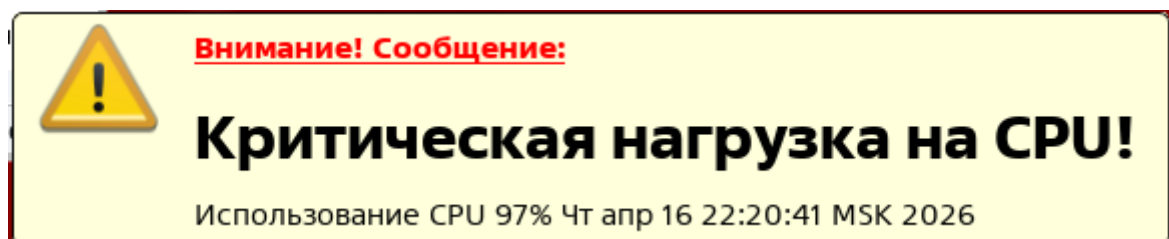


Рисунок 25 – уведомление о критической нагрузке на CPU

Задание 3

1. Измените настройки контрольной группы службы **resource-demanding1.service** так, чтобы суммарная нагрузка на два CPU была в пределах 85-94%. Убедитесь, что сообщение от скрипта мониторинга изменилось.

В файле `/etc/systemd/system/resource-demanding1.service.d/10-CPUSettings.conf`

установлено значение CPUQuota=90% (Рисунок 26). После перезапуска службы в htop видно, что процесс sha256sum потребляет около 86.5% CPU (Рисунок 27). Сообщение мониторинга остаётся критическим (нагрузка выше 85%).

```
/etc/systemd/system/resource-demanding1.service.d/10-CPUSettings.conf

[Service]
CPUQuota=90%
```

Рисунок 26 – установка CPUQuota=90%

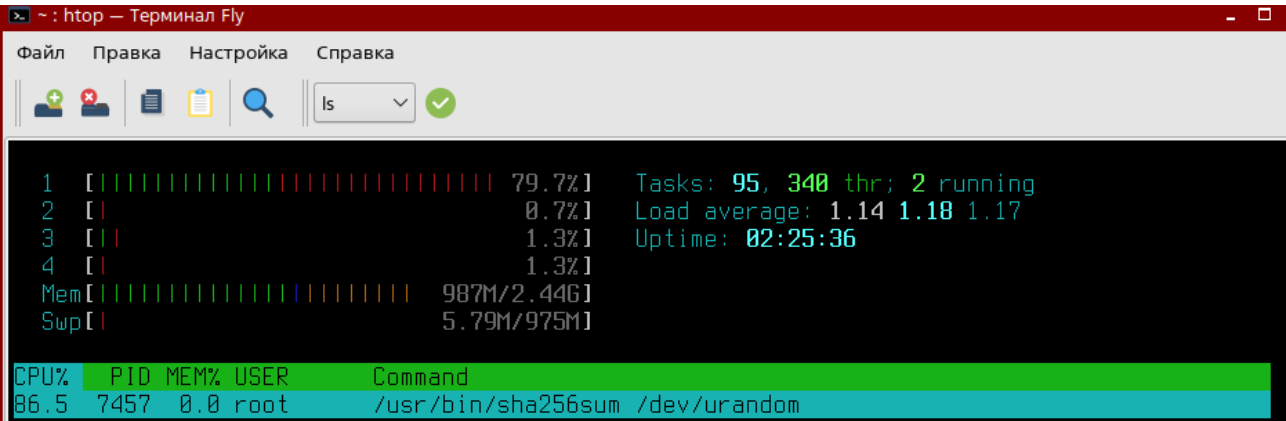


Рисунок 27 – использование CPU после ограничения (86.5%)

2. Измените настройки контрольной группы службы resource-demanding1.service так, чтобы суммарная нагрузка на два CPU была ниже 85%. Убедитесь, что сообщение от скрипта мониторинга изменилось и больше не появляется.

```
/etc/systemd/system/resource-demanding1.service.d/10-CPUSettings.conf

[Service]
CPUQuota=50%
```

Рисунок 28 – установка CPUQuota=50%



Внимание! Сообщение:

Нагрузка на CPU в пределах нормы

Использование CPU 73% Чт апр 16 22:39:20 MSK 2026

Рисунок 29 – уведомление о нормализации нагрузки

3. Выведите на экран дерево контрольных групп и найдите в нем контрольную группу для сервиса resource-demanding1.service.

```

sa@astra:~$ systemd-cgls
Control group /:
-.slice
├─user.slice
│   └─user-1008.slice
│       ├──session-10.scope
│       │   ├──719 /bin/login -p --
│       │   └─2512 -bash
│       └─user@1008.service
│           └─init.scope
│               ├──2502 /lib/systemd/systemd --user
│               └─2503 (sd-pam)
└─session-12.scope
    ├──2585 -:0
    ├──2674 fly-wm
    ├──2739 /usr/bin/dbus-launch --exit-with-session --sh-syntax
    ├──2740 /usr/bin/dbus-daemon --syslog --fork --print-pid 5 --print-address 7 --session
    ├──2764 /usr/bin/kglobalaccel5
    ├──2780 /usr/bin/ssh-agent fly-wm
    ├──2788 /usr/lib/x86_64-linux-gnu/libexec/polkit-kde-authentication-agent-1
    ├──2790 /usr/lib/x86_64-linux-gnu/libexec/org_kde_powerdevil
    ├──2791 qbat
    ├──2792 /usr/bin/fly-reflex-service
    ├──2796 kscreen
    ├──2798 /usr/lib/at-spi2-core/at-spi-bus-launcher --launch-immediately
    ├──2799 fly-search-panel --hidden
    ├──2800 nm-applet
    ├──2804 astra-event-watcher
    ├──2807 /usr/bin/fly-cups-watch
    ├──2810 /usr/lib/gvfs/gvfsd
    ├──2811 fly-brightness
    ├──2822 kmix --keepvisibility
    ├──2834 /usr/bin/dbus-daemon --config-file=/usr/share/defaults/at-spi2/accessibility.c
    └─2843 /usr/bin/pulseaudio --start --log-target=syslog
lines 1-33...skipping...
Control group /:

```

Рисунок 30 – дерево контрольных групп (cgroups)

Задание 4

1. Выведите на экран все запущенные модули подсистемы systemd.

```

sa@astra:~$ systemctl
UNIT                                LOAD    ACTIVE SUB    DESCRIPTION
proc-sys-fs-binfmt_misc.automount  loaded active waiting Arbitrary Executable File
sys-devices-pci0000:00-0000:00:01.1-ata2-host1-target1:0:0-1:0:0:0-block-sr0.device loaded active plugged VBOX_CD-ROM
sys-devices-pci0000:00-0000:00:03.0-net-eth0.device loaded active plugged 82540EM Gigabit Ethernet
sys-devices-pci0000:00-0000:00:05.0-sound-card0.device loaded active plugged 82801AA AC'97 Audio Contr
sys-devices-pci0000:00-0000:00:0d.0-ata1-host0-target0:0:0-0:0:0:0-block-sda-sda1.device loaded active plugged VBOX_HARDDISK 1
sys-devices-pci0000:00-0000:00:0d.0-ata1-host0-target0:0:0-0:0:0:0-block-sda-sda2.device loaded active plugged VBOX_HARDDISK 2
sys-devices-pci0000:00-0000:00:0d.0-ata1-host0-target0:0:0-0:0:0:0-block-sda-sda5.device loaded active plugged VBOX_HARDDISK 5
sys-devices-pci0000:00-0000:00:0d.0-ata1-host0-target0:0:0-0:0:0:0-block-sda.device loaded active plugged VBOX_HARDDISK
sys-devices-platform-serial0250-tty-ttyS0.device loaded active plugged /sys/devices/platform/ser
sys-devices-platform-serial0250-tty-ttyS1.device loaded active plugged /sys/devices/platform/ser

```

Рисунок 31 – список запущенных модулей systemd

2. Выведите на экран все модули служб подсистемы systemd.

```

sa@astra:~$ systemctl list-unit-files --type=service
UNIT FILE                                STATE
-----
acpi-support.service                    disabled
acpid.service                           disabled
alsa-restore.service                    static
alsa-state.service                      static
alsa-utils.service                      masked
anacron.service                         enabled
apt-daily-upgrade.service                static
apt-daily.service                       static
astra-mount-lock.service                 disabled
astra-orientation.service                enabled
auditd.service                          enabled
autovt@.service                         enabled
avahi-daemon.service                    enabled
bootlogd.service                        masked
bootlogs.service                        masked
bootmisc.service                        masked
checkfs.service                         masked
checkroot-bootclean.service              masked
checkroot.service                       masked
clean-mount-point@.service               static
console-getty.service                   disabled
console-setup.service                   enabled
container-getty@.service                 static
cpu-monitor.service                     enabled
cron.service                            enabled
cryptdisks-early.service                 masked
cryptdisks.service                      masked
cups.service                            enabled
dbus-fi.w1.wpa_supplicant1.service       enabled
dbus-org.freedesktop.Avahi.service        enabled
dbus-org.freedesktop.hostname1.service    static
dbus-org.freedesktop.import1.service      static
dbus-org.freedesktop.locale1.service      static
dbus-org.freedesktop.login1.service       static
dbus-org.freedesktop.machine1.service     static

```

Рисунок 32 – список всех модулей служб

3. Выведите на экран в отдельном терминале информацию обо всех поступающих в реальном времени событиях подсистемы systemd. На Рисунке 33 показан вывод событий в реальном времени

```

sa@astra:~$ sudo journalctl -f
-- Logs begin at Thu 2026-04-16 20:07:30 MSK. --
anp 16 22:46:45 astra sudo[9313]: pam_unix(sudo:session): session closed for user root
anp 16 22:46:45 astra audit[9313]: USER_END pid=9313 uid=0 auid=1008 ses=12 subj=0:63:0:0 msg=
'op=PAM:session_close grantors=pam_permit,pam_unix acct="root" exe="/usr/bin/sudo" hostname=
? addr=? terminal=/dev/pts/0 res=success'
anp 16 22:46:45 astra audit[9313]: CRED_DISP pid=9313 uid=0 auid=1008 ses=12 subj=0:63:0:0 ms
g='op=PAM:setcred grantors=pam_permit acct="root" exe="/usr/bin/sudo" hostname=? addr=? termi
nal=/dev/pts/0 res=success'
anp 16 22:46:46 astra audit[9323]: USER_ACCT pid=9323 uid=1008 auid=1008 ses=12 subj=0:63:0:0
msg='op=PAM:accounting grantors=pam_tally,pam_permit acct="sa" exe="/usr/bin/sudo" hostname=
? addr=? terminal=/dev/pts/0 res=success'
anp 16 22:46:46 astra audit[9323]: USER_CMD pid=9323 uid=1008 auid=1008 ses=12 subj=0:63:0:0
msg='cwd="/home/sa" cmd=6A6F75726E616C63746C202D066 terminal=pts/0 res=success'
anp 16 22:46:46 astra sudo[9323]:          sa : TTY=pts/0 ; PWD=/home/sa ; USER=root ; COMMAND=/
usr/bin/journalctl -f
anp 16 22:46:46 astra audit[9323]: CRED_REFR pid=9323 uid=0 auid=1008 ses=12 subj=0:63:0:0 ms
g='op=PAM:setcred grantors=pam_permit acct="root" exe="/usr/bin/sudo" hostname=? addr=? termi
nal=/dev/pts/0 res=success'
anp 16 22:46:46 astra audit[9323]: USER_START pid=9323 uid=0 auid=1008 ses=12 subj=0:63:0:0 m
sg='op=PAM:session_open grantors=pam_permit,pam_unix acct="root" exe="/usr/bin/sudo" hostname
=? addr=? terminal=/dev/pts/0 res=success'
anp 16 22:46:46 astra sudo[9323]: pam_kiosk2(sudo:session): need_continue: UID 0 detected, sk
ipping. User: root
anp 16 22:46:46 astra sudo[9323]: pam_unix(sudo:session): session opened for user root by sa(
uid=0)
^C
sa@astra:~$

```

Рисунок 33 – журнал событий в реальном времени

4. Перезапустите службу **resource-demanding1.service**. Выведите на экран её статус.

На Рисунке 34 показан статус: служба активна, с указанием PID и дочерних процессов.

```

sa@astra:~$ sudo systemctl restart resource-demanding1.service
sa@astra:~$ systemctl status resource-demanding1.service
● resource-demanding1.service - Сервис, воспроизводящий высокую нагрузку на CPU
   Loaded: loaded (/etc/systemd/system/resource-demanding1.service; disabled; vendor preset:
   Drop-In: /etc/systemd/system/resource-demanding1.service.d
            └─00-slice.conf, 10-CPUSettings.conf
   Active: active (running) since Thu 2026-04-16 22:47:36 MSK; 11s ago
   Main PID: 9374 (bash)
     Tasks: 2 (limit: 2889)
    Memory: 600.0K
       CPU: 5.848s
    CGroup: /system.slice/resource-demanding1.service
            └─9374 /usr/bin/bash /usr/local/bin/resource-demanding1.sh
              └─9375 /usr/bin/sha256sum /dev/urandom
lines 1-12/12 (END)

```

Рисунок 34 – статус службы после перезапуска

5. Остановите службы **resource-demanding1.service**, **resource-demanding2.service** и **cpu-monitoring.service**.

```

sa@astra:~$ sudo systemctl stop resource-demanding1.service resource-demanding2.service
onitor.service
sa@astra:~$

```

Рисунок 35 – остановка служб

6. Выведите на экран информацию из журналов о сервисе resource-demanding1.service в сокращённом виде.

Рисунке 36 показан вывод журнала: время запусков, завершений, код выхода и потребление CPU.

```
sa@astra:~$ sudo journalctl -u resource-demanding1.service -o short
-- Logs begin at Thu 2026-04-16 20:07:30 MSK, end at Thu 2026-04-16 22:52:02 MSK. --
apr 16 21:42:26 astra systemd[1]: Started Сервис, Воспроизводящий Высокую нагрузку на CPU.
apr 16 22:03:13 astra taskset[3691]: Завершено
apr 16 22:03:13 astra systemd[1]: resource-demanding1.service: Main process exited, code=exited, status=143/n/a
apr 16 22:03:13 astra systemd[1]: resource-demanding1.service: Failed with result 'exit-code'.
apr 16 22:03:13 astra systemd[1]: resource-demanding1.service: Consumed 18min 41.179s CPU time.
apr 16 22:19:12 astra systemd[1]: Started Сервис, Воспроизводящий Высокую нагрузку на CPU.
apr 16 22:21:25 astra taskset[6085]: Завершено
apr 16 22:21:25 astra systemd[1]: resource-demanding1.service: Main process exited, code=exited, status=143/n/a
apr 16 22:21:25 astra systemd[1]: resource-demanding1.service: Failed with result 'exit-code'.
apr 16 22:21:25 astra systemd[1]: resource-demanding1.service: Consumed 1min 54.785s CPU time.
apr 16 22:22:48 astra systemd[1]: Started Сервис, Воспроизводящий Высокую нагрузку на CPU.
apr 16 22:26:31 astra taskset[6837]: Завершено
apr 16 22:26:31 astra systemd[1]: resource-demanding1.service: Main process exited, code=exited, status=143/n/a
apr 16 22:26:31 astra systemd[1]: resource-demanding1.service: Failed with result 'exit-code'.
apr 16 22:26:31 astra systemd[1]: resource-demanding1.service: Consumed 3min 21.076s CPU time.
apr 16 22:29:52 astra systemd[1]: Started Сервис, Воспроизводящий Высокую нагрузку на CPU.
apr 16 22:31:04 astra systemd[1]: Stopping Сервис, Воспроизводящий Высокую нагрузку на CPU...
apr 16 22:31:04 astra systemd[1]: resource-demanding1.service: Main process exited, code=killed, status=15/TERM
apr 16 22:31:04 astra systemd[1]: resource-demanding1.service: Succeeded.
apr 16 22:31:04 astra systemd[1]: Stopped Сервис, Воспроизводящий Высокую нагрузку на CPU.
apr 16 22:31:04 astra systemd[1]: resource-demanding1.service: Consumed 47.638s CPU time.
apr 16 22:31:04 astra systemd[1]: Started Сервис, Воспроизводящий Высокую нагрузку на CPU.
apr 16 22:32:14 astra systemd[1]: Stopping Сервис, Воспроизводящий Высокую нагрузку на CPU...
apr 16 22:32:14 astra systemd[1]: resource-demanding1.service: Main process exited, code=killed, status=15/TERM
apr 16 22:32:14 astra systemd[1]: resource-demanding1.service: Succeeded.
apr 16 22:32:14 astra systemd[1]: Stopped Сервис, Воспроизводящий Высокую нагрузку на CPU.
apr 16 22:32:14 astra systemd[1]: resource-demanding1.service: Consumed 1min 2.199s CPU time.
```

Рисунок 36 – журнал службы resource-demanding1.service в сокращённом формате

Вопросы

1. **Какое сочетание технологий рекомендуется использовать в современных системах?**

Ответ: UEFI + GPT (стр. 10: «Сейчас такой способ загрузки считается наиболее правильным и настоятельно рекомендуется всеми разработчиками операционных систем»).

2. **Расположите пункты запуска компьютера с ОС Linux от момента включения до момента запуска ядра в правильном порядке:**

Ответ:

- b. Тестирование и инициализация оборудования
- c. Поиск загрузочного устройства
- e. Запуск загрузчика первого этапа
- g. Запуск загрузчика второго этапа
- f. Вывод загрузочного меню
- d. Загрузка ядра

(стр. 1–9, описано в процессе загрузки)

3. **Расположите пункты запуска компьютера с ОС Linux от момента запуска ядра до момента загрузки системы в выбранном целевом состоянии:**

Ответ:

- c. Инициализация наиболее важных функций ядра
- b. Монтирование `init ram fs` в корень
- a. Загрузка модулей ядра и драйверов с RAM-диска
- f. Запуск скрипта `/init`
- e. Монтирование устройства с системным диском в корень
- d. Запуск процесса `systemd`
- g. Запуск модулей `systemd`

(стр. 18–20)

4. **В каком файле следует выполнять настройки GRUB?**

Ответ: В файле `/etc/default/grub` (стр. 15–16).

5. **Какая команда позволит применить новую конфигурацию GRUB?**

Ответ: `update-grub` (или `grub-mkconfig -o /boot/grub/grub.cfg`) (стр. 16).

6. **Какого модуля `systemd` не существует?**

Ответ: В тексте перечислены

модули: `.target`, `.service`, `.mount`, `.automount`, `.swap`, `.timer`, `.socket`, `.slice`, `.device`, `.path`, `.snapshot`.

Например, нет модуля `.user` (стр. 21–22). Наиболее явно не существует модуля, не упомянутого в списке.

7. **Какие целевые состояния ОС соответствуют следующим уровням загрузки?**

- A. runlevel 0 → **poweroff.target**
- B. runlevel 1 → **rescue.target**
- C. runlevel 2 → **multi-user.target**
- D. runlevel 3 → **multi-user.target**
- E. runlevel 4 → **multi-user.target**

F. runlevel 5 → **graphical.target**

G. runlevel 6 → **reboot.target**

(стр. 24)

8. **Какие секции могут присутствовать в конфигурационном файле модуля службы systemd:**

Ответ: [Unit], [Service], [Install] (стр. 26).

9. **Какой командой можно отслеживать события со службами systemd в реальном времени?**

Ответ: `sudo journalctl -f` (стр. 42).

10. **Какая команда выведет список всех служб и их состояния:**

Ответ: `systemctl list-unit-files --type=service` (стр. 40).